

INSIDE / DIST

IF-ELSE / IMPLICATION

SOFT / UNIQUE

SOLVE...BEFORE

SystemVerilog

Constraints

inside · dist · soft · unique · solve...before · randcase

A practical SystemVerilog reference for
constrained-random verification — inside, dist,
conditional and implication constraints, soft, unique,
solve...before, inheritance, and randcase.

FILE TYPE	SystemVerilog Constraints Reference, Beginner to Advanced
TOPICS	inside · dist · if-else · implication · foreach · soft · unique
INCLUDES	dynamic arrays, inheritance, solve order & randcase examples
TARGET	SystemVerilog / Verification Beginners

AUTHOR

GADDAM VINOD KUMAR · github.com/vinodgaddam1


```
}  
endclass
```

```
module top;  
  sample s1;  
  initial begin  
    repeat (10) begin  
      s1=new();  
      s1.randomize();  
      s1.print();  
    end  
  end  
endmodule
```

//2.Distributive constraint here we using dist key word "dist" with values

```
class sample;  
  rand int a;  
  
  function void print();  
    $display("a = %0d",a);  
  endfunction  
  
  constraint sa{
```

```
a dist {10:=20,100:=2,1000:=5};  
}  
endclass
```

```
module top;  
sample s1;  
initial begin  
repeat (20) begin  
s1=new();  
s1.randomize();  
s1.print();  
end  
end  
endmodule
```

//2.Distrubutive constraint here we using dist key word "dist" with range

```
class sample;  
rand int a;  
  
function void print();  
$display("a = %0d",a);  
endfunction
```

```
constraint sa{  
a dist {[10:30]:/2,[100:200]:/4,[1000:2000]:/6};  
}  
endclass
```

```
module top;  
sample s1;  
initial begin  
repeat (20) begin  
s1=new();  
s1.randomize();  
s1.print();  
end  
end  
endmodule
```

//2.Distributive constraint here we using dist key word "dist" with value + range

```
class sample;  
rand int a;  
  
function void print();  
$display("a = %0d",a);  
endfunction
```

```
constraint sa{  
a dist {[10:30]:/2,200:=9,[1000:2000]:/4};  
}  
endclass
```

```
module top;  
sample s1;  
initial begin  
repeat (20) begin  
s1=new();  
s1.randomize();  
s1.print();  
end  
end  
endmodule
```

//2.Distrubutive constraint here we using dist key word

```
rand int a;
```

```
function void print();  
$display("a = %0d",a);  
endfunction
```

```
constraint sa{  
  a inside {[0:100],[0:100]};  
}  
endclass
```

```
module top;  
  sample s1;  
  initial begin
```

```
    repeat (20) begin  
      s1=new();  
      s1.randomize();  
      s1.print();  
    end  
  end  
endmodule
```

//3.if-else constraint question-1

```
class sample;  
  rand int a,b,c;
```

```
  function void print();  
    $display("a = %0d",a);
```

```
$display("b = %0d",b);  
endfunction
```

```
constraint sa{  
a inside {[-10:10]};  
b inside {[-10:10]};
```

```
if (a>0) b>0;  
else if (a==0) b==0;  
else b<0;  
}  
endclass
```

```
module top;  
sample s1;  
initial begin  
repeat (10) begin  
s1=new();  
s1.randomize();  
s1.print();  
end  
end  
endmodule
```

//3.if-else constraint question-2

```
typedef enum bit [3:0] {SAML, MEDIUM, LARGE, VERY_LARGE} pkt_type;
```

```
class sample;
```

```
rand pkt_type p;
```

```
rand int len;
```

```
function void print();
```

```
$display("pkt_type= %s ,len = %0d",p,len);
```

```
endfunction
```

```
constraint sa{
```

```
    if (p==SAML)
```

```
        len inside {[10:100]};
```

```
    else if (p==MEDIUM)
```

```
        len inside {[101:200]};
```

```
    else if (p==LARGE)
```

```
        len inside {[201:300]};
```

```
    else
```

```
        len == 500;
```

```
}  
endclass  
  
module top;  
sample s1;  
initial begin  
repeat (10) begin  
s1=new();  
s1.randomize();  
s1.print();  
end  
end  
endmodule
```

//4.Implicit Constraint

```
typedef enum bit [3:0] {SAML, MEDIUM, LARGE, VERY_LARGE  
}pkt_type;  
class sample;  
rand pkt_type p;  
rand int len;
```

```
function void print();
$display("pkt_type= %s ,len = %0d",p,len);
endfunction
```

```
constraint sa{
    (p==SMALL) -> (len inside {[10:100]});
    (p==MEDIUM) -> (len inside {[101:200]});
    (p==LARGE) -> len inside {[201:300]};
    (p==VERY_LARGE) -> (len == 500);
}
endclass
```

```
module top;
sample s1;
initial begin
repeat (10) begin
s1=new();
s1.randomize();
s1.print();
end
end
```

```
endmodule
```

```
//5.Iterative variable 0,1,2,3,4,5,6,....
```

```
class sample;
```

```
rand int array[0:9];
```

```
function void print();
```

```
$display("array = %p",array);
```

```
endfunction
```

```
constraint sa{
```

```
foreach(array[i])
```

```
{
```

```
    array[i]==i;
```

```
}
```

```
}
```

```
endclass
```

```
module top;
```

```
sample s1;
```

```
initial begin
```

```
repeat (5) begin
```

```
s1=new();
```

```
s1.randomize();
```

```
s1.print();  
end  
end  
endmodule
```

//5.Iterative variable odd to even & even to odd

```
class sample;  
rand int array[0:9];  
function void print();  
$display("array = %p",array);  
endfunction
```

```
constraint sa{  
    foreach(array[i]){  
        array[i] inside {[10:100]};  
  
        if(i%2==0)  
            array[i]%2==1;  
        else  
            array[i]%2==0;  
  
    }  
}  
endclass
```

```
module top;
sample s1;
initial begin
repeat (1) begin
s1=new();
s1.randomize();
s1.print();
end
end
endmodule
```

//5.Iterative variable 0,1,0,2,0,3.....

```
class sample;
rand int array[0:9];
function void print();
$display("array = %p",array);
endfunction
```

```
constraint sa{
foreach(array[i]){
array[i] inside {[0:10]};
```

```
if(i%2==0)
    array[i]==0;
else
    array[i]==(i+1)/2;

}
}
endclass
```

```
module top;
sample s1;
initial begin
repeat (1) begin
s1=new();
s1.randomize();
s1.print();
end
end
endmodule
```

//5.Iterative variable 0,0,1,0,0,2,0,0,3.....

```
class sample;
rand int array[0:9];
function void print();
```

```
[$display("array = %p",array);  
endfunction
```

```
constraint sa{  
foreach(array[i]){  
array[i] inside {[0:10]};
```

```
if((i+1)%3==0)  
    array[i]==(i+1)/3;  
else  
    array[i]==0;
```

```
}
```

```
}
```

```
endclass
```

```
module top;  
sample s1;  
initial begin  
repeat (1) begin  
s1=new();  
s1.randomize();  
s1.print();  
end
```

end

endmodule

//5.Iterative variable 0,9,99,999,9999,99999,....

class sample;

rand int array[0:9];

function void print();

\$display("array = %p",array);

endfunction

constraint sa{

foreach(array[i]){

array[i]==(10**i)-1;

}

}

endclass

module top;

sample s1;

initial begin

```
repeat (1) begin
s1=new();
s1.randomize();
s1.print();
end
end
endmodule
```

//5.Iterative variable 2,5,7,4,10,14,6,..

```
class sample;
rand int array[0:9];
function void print();
$display("array = %p",array);
endfunction
```

```
constraint sa{
foreach(array[i]){
if (i%3==0)
array[i]==2*(i+3)/3;
else if ((i-1)%3==0)
array[i]==5*(i+2)/3;
else
array[i]==7*(i+1)/3;
```

```
}  
}  
endclass  
  
module top;  
  sample s1;  
  initial begin  
    repeat (1) begin  
      s1=new();  
      s1.randomize();  
      s1.print();  
    end  
  end  
endmodule
```

//5.Iterative variable unique values

```
class sample;  
  rand int array[0:9];  
  function void print();  
    $display("array = %p",array);  
endfunction
```

```
constraint sa{
foreach(array[i]){
    array[i] inside {[10:20]};
    foreach(array[j])
        if(i!=j)
            array[i]!=array[j];
}
}
endclass
```

```
module top;
sample s1;
initial begin
repeat (1) begin
s1=new();
s1.randomize();
s1.print();
end
end
endmodule
```

// Noted: dynamic array we can allocate 4 ways

//1.In function

```
class sample;  
rand int array [];//dynamic array  
function void print();  
$display("array = %p",array);  
endfunction  
function new();  
    array=new[10];  
endfunction
```

```
constraint sa{  
foreach(array[i]){  
array[i]==i*2;  
}  
}  
endclass
```

```
module top;  
sample s1;  
initial begin  
repeat (1) begin  
s1=new();  
s1.randomize();  
s1.print();  
end
```

```
end  
endmodule
```

```
// Noted: dynamic array we can allocate 4 ways
```

```
//2.pre_randomization
```

```
class sample;  
  rand int array [];//dynamic array  
  function void print();  
  $display("array = %p",array);  
endfunction  
  function void pre_rand1();  
    array=new[10];  
endfunction
```

```
constraint sa{  
  foreach(array[i]){  
    array[i]==(i*2)+1;  
  }  
}  
endclass
```

```
module top;
sample s1;
initial begin
repeat (1) begin
s1=new();
s1.pre_rand1();
s1.randomize();
s1.print();
end
end
endmodule
```

// Noted: dynamic array we can allocate 4 ways

//3.In constraint

```
class sample;
rand int array [];//dynamic array
function void print();
$display("array = %p",array);
endfunction

constraint sa{
    array.size==10;
foreach(array[i]){
```

```
array[i]==(i*10)+1;  
}  
}  
endclass
```

```
module top;  
sample s1;  
initial begin  
repeat (1) begin  
s1=new();  
s1.randomize();  
s1.print();  
end  
end  
endmodule
```

// Noted: dynamic array we can allocate 4 ways

//4.In module

```
class sample;  
rand int array [];//dynamic array  
function void print();  
$display("array = %p",array);  
endfunction
```

```
constraint sa{
foreach(array[i]){
array[i]==(i*10)+1;
}
}
endclass
```

```
module top;
sample s1;
initial begin
repeat (1) begin
s1=new();
s1.array=new[10];
s1.randomize();
s1.print();
end
end
endmodule
```

//5.Iterative variable consecutive numbers '1' like--> way

010101/001001

```
class sample;
```

```
rand bit [0:15]vec;  
function void print();  
$display("Vector = %b",vec);  
endfunction
```

```
constraint sa{  
    foreach(vec[i]){  
        if(i>0)  
            if(vec[i-1]==1)  
                vec[i]!=1;  
    }  
}  
endclass
```

```
module top;  
    sample s1;  
    initial begin  
        repeat (20) begin  
            s1=new();  
            s1.randomize();  
            s1.print();  
        end  
    end  
end
```

```
endmodule
```

```
//queue
```

```
class sample;
```

```
rand int array [$];
```

```
function void print();
```

```
$display("array = %p",array);
```

```
endfunction
```

```
constraint sa{
```

```
    array.size inside {[5:15]};
```

```
foreach(array[i]){
```

```
    array[i] inside {[10:50]};
```

```
}
```

```
}
```

```
endclass
```

```
module top;
```

```
sample s1;
```

```
initial begin
```

```
repeat (1) begin
```

```
s1=new();
```

```
s1.array=new[10];
```

```
s1.randomize();  
s1.print();  
end  
end  
endmodule
```

```
//7.soft constrain
```

```
class sample;  
    rand int a;  
    function void print();  
        $display("a=%0d",a);  
    endfunction  
  
    constraint ac{  
        soft a inside {[10:20]};  
    }  
endclass
```

```
module top;  
    sample s;  
    initial begin  
        repeat(10)begin
```

```

    s=new();
//    s.randomize()with{a==100;};Constrant conflict
//    s.randomize()with{soft a==100;}; Inline is soft
//    s.randomize()with{a==100;}; Inclass is soft
    s.randomize()with{soft a==100;};// Inclass && Inline both are
soft taht time inline is more weighit
    s.print();
end
end
endmodule

```

//8.unique constrain

```

class sample;
    rand int a,b,c;

    function void print ();
        $display("a=%0d",a);
        $display("b=%0d",b);
        $display("c=%0d",c);
    endfunction

    constraint ac{

```

```

        a inside {[10:100]};
        b inside {[10:200]};
        c inside {[10:300]};
        unique{a,b,c};

    }
endclass

module top;
sample s;
initial begin
    repeat (20) begin
s=new();
s.randomize();
s.print();
end
end
endmodule

```

//9.Variable constrain

```

class sample;
    rand int a,b;

    function void print ();

```

```
        $display("a=%0d",a);
        $display("b=%0d",b);
    endfunction
```

```
    constraint ac{
        (b!=0) -> (a==0);
        solve b before a;
```

```
    }
```

```
endclass
```

```
module top;
```

```
    sample s;
```

```
    initial begin
```

```
        repeat (20) begin
```

```
            s=new();
```

```
            s.randomize();
```

```
            s.print();
```

```
        end
```

```
    end
```

```
endmodule
```

//constraint support oops concept or not

```
class parent;
    rand int a;

    function void print();
        $display("a = %03d",a);
    endfunction

    constraint a_c{
        a inside {[10:100]};
    }
endclass
```

```
class child extends parent;
    rand int b;

    function void print();
        super.print();
        $display("b=%03d",b);

    endfunction

    constraint a_c{
```

```
    b inside {[1000:2000]};  
}
```

```
endclass
```

```
module top;
```

```
  child c;
```

```
  initial begin
```

```
    c=new();
```

```
    c.randomize();
```

```
    c.print();
```

```
  end
```

```
endmodule
```

```
//constraint support oops concept or not
```

```
class parent;
```

```
  rand int a;
```

```
  virtual function void print();
```

```
    $display("parent class %0d",a);
```

```
endfunction
```

```
        constraint a_c{
            a inside {[10:100]};
        }
endclass
```

```
class child extends parent;
    rand int b;
```

```
    function void print();
        super.print();
        $display("child class %0d",b);
```

```
endfunction
```

```
        constraint a_c{
            b inside {[1000:2000]};
        }

endclass
```

```
module top;
    parent p;
    child c;
    initial begin
        c=new();
```

```
    c.randomize();
    c.print();
    p=c;
    p.print();
end
endmodule
```

//constraint be override or not

```
class parent;
    rand int a;

    function void print();
        $display("a = %03d",a);
    endfunction

    constraint a_c{
        a inside {[10:20]};
    }
endclass
```

```
class child extends parent;
    rand int b;
```

```
function void print();  
    super.print();  
    $display("b=%03d",b);
```

```
endfunction
```

```
constraint b_c{  
    b inside {[100:200]};  
}
```

```
constraint a_c{  
    a inside {[1000:2000]};  
}
```

```
endclass
```

```
module top;
```

```
    child c;
```

```
    initial begin
```

```
        c=new();
```

```
        c.randomize();
```

```
        c.print();
```

```
    end
```

```
endmodule
```

```
// randcase
```

```
class trans;
```

```
rand int a;
```

```
function void print();
```

```
$display("a=%0d",a);
```

```
endfunction
```

```
endclass
```

```
class sample;
```

```
trans tx;
```

```
    task run();
```

```
    randcase
```

```
        1:begin
```

```
            tx=new();
```

```
            tx.randomize() with {a inside{[10:20]}};
```

```
            tx.print();
```

```
        end
```

```
        5:begin
```

```
            tx=new();
```

```
            tx.randomize() with {a inside{[100:200]}};
```

```
            tx.print();
```

```
        end
```

```
        4:begin
```

```
            tx=new();
```

```

        tx.randomize() with {a inside {[1000:2000]}};};
    tx.print();
end
endcase
endtask

endclass

module top;
sample s;
initial begin
    repeat(20)begin
        s= new();
        s.run();
    end
end
endmodule

```

```

//Range constraint

```

```

class sample;
rand int a;
function void print();
    $display("a=%0d",a);
endfunction

```

```
constraint a_c{
    a>=10;
    a<=100; //This is equal to a inside {[10:100]};
}
endclass
```

```
module top;
sample s;
initial begin
    repeat (10)begin
        s=new();
        s.randomize();
        s.print;
    end
end
endmodule
```

//Multipul ranges constraint

```
class sample;
rand int opcode;
function void print();
    $display("opcode=%0d",opcode);
endfunction
```

```
constraint a_c{  
    opcode inside {[1:4],[8:12]};  
}  
endclass
```

```
module top;  
sample s;  
initial begin  
    repeat (10)begin  
        s=new();  
        s.randomize();  
        s.print;  
    end  
end  
endmodule
```